Lab 3: Component Modelling & Architectural Pattern Selection

Self-Service Coffee Kiosk System - Architectural Justification

Name: Ronak Thamaran | SRN: PES2UG24CS410 | Section: G (CSE)

Architecture Selection:

We chose Layered Architecture for the Self-Service Coffee Kiosk System.

Reason 1: Clear Separation of Concerns

The kiosk has three clearly separate concerns: handling touch-screen interaction, processing orders/payments/receipts, and storing menu and pricing data. A layered design maps each concern to its own layer (Presentation, Business, Data), so the UI can change without affecting order or payment logic. This separation ensures that modifications to the touchscreen interface (e.g., updating the layout or adding animations) do not require changes to the core business logic or database structure.

Reason 2: Appropriate Scale for Single-Device Deployment

The kiosk is a small, single-purpose system running on one device, not something that needs to scale independently across a network. Microservices would add unnecessary network overhead and operational complexity for this scale, while plain Client-Server does not on its own separate UI, logic, and data the way this system needs. Layering gives that internal structure while keeping deployment simple—all components run within the same process on the kiosk hardware, eliminating inter-service communication latency and reducing the risk of network-related failures during busy periods.

Security Advantage: Controlled Access to Sensitive Operations

In a layered design, the Presentation Layer cannot access the Database directly—all requests go through the Order Manager in the Business Layer. This keeps sensitive operations like credit card payment processing and pricing data behind a controlled interface, away from the touch-screen front end. By enforcing this separation, the architecture prevents unauthorized direct manipulation of pricing or payment data, reducing attack vectors. Any attempt to bypass the business logic (e.g., tampering with prices at the UI level) is blocked by the centralized validation and authorization enforced in the Business Layer.

Performance Benefit: Low Latency and In-Process Communication

Since all components run on the same kiosk device, layering avoids the network latency that a microservices approach would introduce between services. The Business Layer can also cache menu and pricing data from the Database, keeping order placement fast during busy periods. In-process method calls between layers execute in microseconds, ensuring the kiosk responds instantly to customer interactions. This is critical during peak hours when multiple customers may be using adjacent kiosks—low latency ensures a smooth user experience and prevents queue buildup.

Components Identified

1. **User Interface Component** - Handles touch-screen interaction, provides OrderInput interface to customers.
2. **Order Manager Component** - Coordinates orders, provides OrderInterface, requires PaymentInterface, PrintInterface, and DBInterface.
3. **Payment Service Component** - Processes card payments, provides PaymentAPI for secure card authorization and transaction settlement.
4. **Receipt Printer Driver Component** - Drives the receipt printer hardware, provides PrintService for ESC/POS formatted receipts.
5. **Menu & Pricing Database Component** - Stores menu data (3 coffee types: Espresso, Americano, Latte) and pricing for 2 sizes (Small, Large), provides DBInterface.

Component Diagram:

